

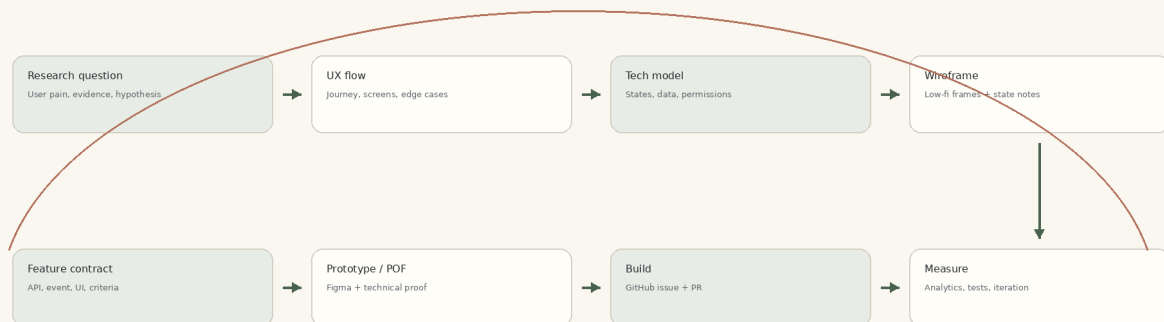
EduHackers

Developer ↔ UI/UX Workflow + Wireframe Research Dossier

How the Developer / Research Lead and UX/UI Designer should work together to turn the Eduvos marketplace concept into buildable flows, contracts, wireframes, APIs and implementation tasks.

Prepared for: EduHackers team review | Audience: Developer, UX/UI Designer, Product Lead, Business Analyst, Project Manager / Pitch Lead | Status: research hypotheses and recommended workflow, not final production specification.

Developer ↔ UI/UX workflow: shared research-to-build loop



UI/UX owns
● user journey ● wireframes ● Figma prototype ● component states ● accessibility

Developer owns
● state machine ● API/event contracts ● data model ● POF tests ● security boundaries

Shared owns
● feature contract ● decision log ● acceptance criteria ● usability findings ● release checklist

1. Executive summary

This dossier translates the developer-side architecture into a working collaboration system between the Developer / Research Lead and the UX/UI workstream. The core recommendation is not a traditional handoff. EduHackers should operate a dual-track workflow where research, UX flows, wireframes, technical state machines, API contracts and GitHub implementation tasks are produced together. The team should avoid building screens that do not have backend states, permissions, error states and acceptance criteria attached.

The output also includes low-fidelity example wireframes for Marketplace and Operations Platform screens. These examples are not final visual design; they are system wireframes showing how buyer, seller, courier, hub operator, transfer, moderation and incubation flows could connect.

Strategic recommendation

Use Miro for research/system thinking, Figma for wireframes/prototypes/components, and GitHub for feature contracts, API definitions, implementation tasks, pull requests and decision logs. Every important flow should move through a shared Feature Contract before build.

Evidence status

The workflow recommendations are grounded in official tool capabilities and product-engineering patterns. Figma Dev Mode supports inspecting designs, ready-for-dev status, annotations and developer integrations; Code Connect can map design components to code components. GitHub Issues and Projects link planning directly to issues and pull requests. OpenAPI provides a standard interface description for REST APIs. WCAG 2.2 provides a widely used accessibility framework. These are used as references, not as claims that the team already has paid seats or institutional integrations. [R1-R8]

2. Why this workflow matters for EduHackers

EduHackers is not a simple static marketplace. The system includes real-time seller dispatch, courier dispatch, package custody, hub scanning, cross-campus transfers, role-based operations, seller capacity, protected payments, wallet/withdrawals, disputes, and Incubation Hub intelligence. If UI/UX designs screens in isolation, the prototype can look attractive but fail the moment a seller declines, a courier times out, a package is rejected or a payment state changes.

The Developer and UX/UI Designer must therefore share one mental model: every visible state on screen must be traceable to a backend state or event. Every backend state must be represented clearly enough that the user knows what is happening, what they can do next, and what the platform is doing on their behalf.

The simplest rule: no screen is considered ready unless it has a matching state/API/event contract and acceptance criteria.

What breaks without this workflow

- Designs can include buttons that do not map to any real backend transition.
- Developers can build endpoints that do not correspond to understandable user actions.
- Edge cases such as timeout, decline, hub rejection, courier unavailability and refund states get handled too late.
- Marketplace trust can degrade because the UI does not explain protected payment, custody or verification clearly.
- The team loses traceability between research evidence, product decisions, Figma screens and GitHub build tasks.

3. Collaboration system overview

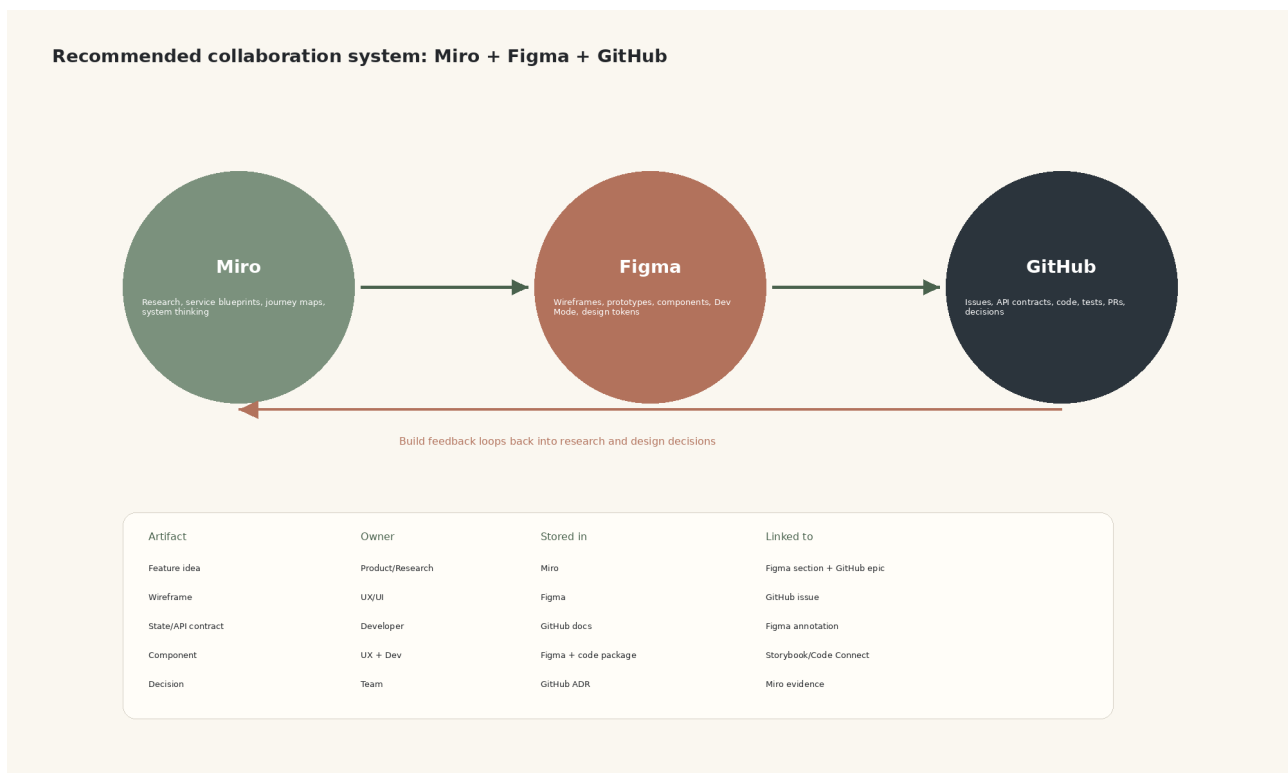


Figure 1. Recommended tool system. Miro, Figma and GitHub each have a clear role. They should be connected by links and shared feature contracts, not treated as isolated workspaces.

The collaboration system should have three source-of-truth layers. Miro stores early research synthesis, maps, service blueprints and unresolved decisions. Figma stores wireframes, prototypes, component states and developer annotations. GitHub stores issues, API contracts, source code, tests and architecture decision records. Large video files, reference PDFs and heavy design exports should not be treated as normal Git history.

4. Developer ↔ UI/UX operating model

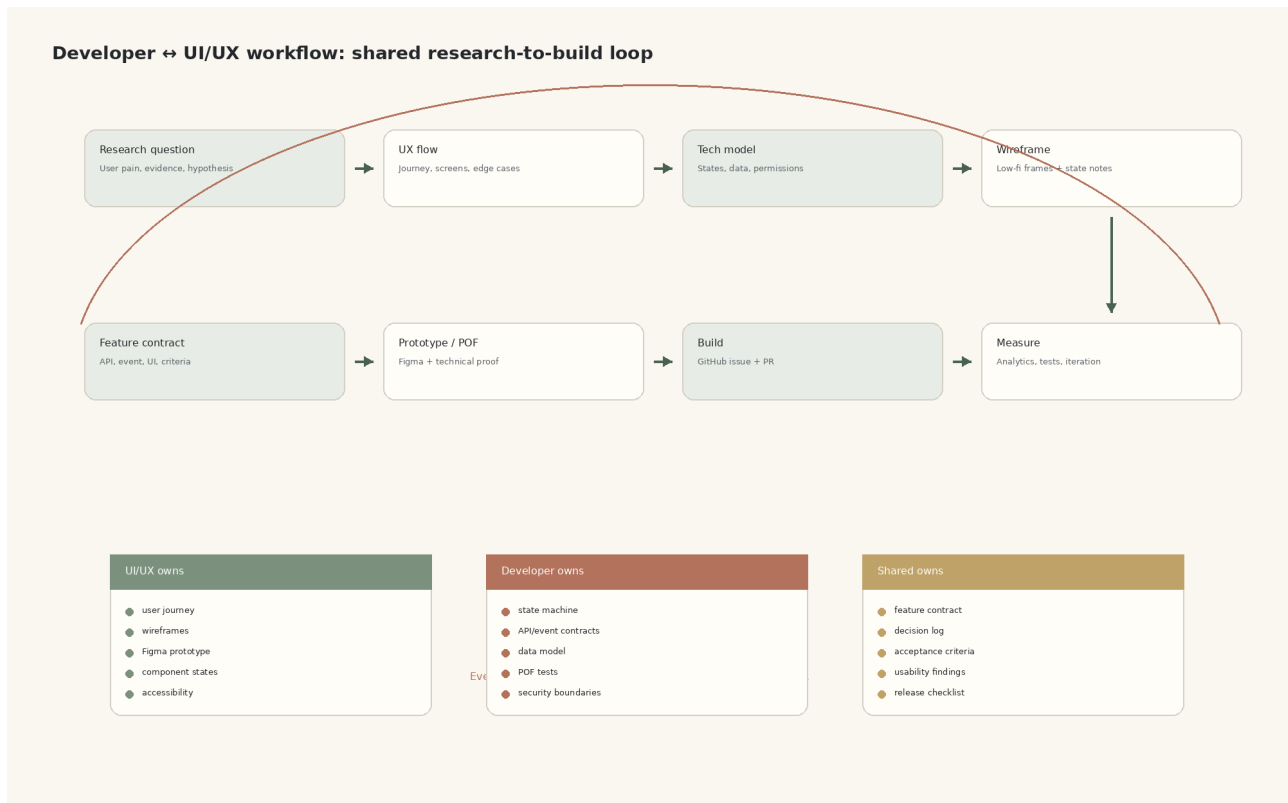


Figure 2. Research-to-build workflow loop. The workflow intentionally returns build findings back into research and design rather than treating development as the end of the process.

The Developer / Research Lead and the UX/UI Designer should work as a pair during the first pass of every high-risk flow. This does not mean both people do the same work. It means UX flow decisions and technical state decisions are made close together. The Developer owns feasibility, state machines, APIs, data and security boundaries. The UX/UI Designer owns the interaction model, screen flow, information hierarchy, accessibility and Figma prototype. Both own the Feature Contract.

5. Feature Contract: the required handoff artifact

Feature Contract: the bridge between Figma and GitHub

Backend state	UI state	User action	API / event	Next state
SELLER_OFFERED	Incoming request card + countdown	Accept	POST /offers/{id}/accept	SELLER_MATCHED
PAYMENT_PENDING	Checkout timer + exact total	Pay	payment.succeeded callback	PAID
HUB_REJECTED	Package rejected + 24h deadline	Repackage	hub.package.rescan	HUB_RECEIVED
COURIER_ASSIGNED	Courier card, ETA, call/message	Track	ws:courier.location	IN_TRANSIT
TRANSFER_EXCEPTION	Discrepancy review panel	Submit info	POST /cases/{id}/statement	INVESTIGATION_OPEN

Rule: no screen is ready for development until its backend states, events, errors, permissions and acceptance criteria are mapped.

Figure 3. Feature Contract matrix. Each row connects backend state, UI state, user action, API/event and next state.

The Feature Contract is the main bridge between Figma and GitHub. It should be short enough to maintain but strict enough to prevent guessing. The contract should be attached to the GitHub issue and linked from the relevant Figma frame or section.

Feature Contract fields

- Feature name and user goal.
- User role: buyer, seller, courier, hub operator, transfer operator, moderator, incubation staff, admin or finance.
- Entry points and previous states.
- Backend states represented by the screen.
- Primary action and secondary actions.
- REST endpoints, WebSocket events and notifications involved.
- Permissions and role checks.
- Loading, empty, timeout, offline and error states.
- Analytics events and success metrics.
- Accessibility requirements and acceptance criteria.

6. Weekly workflow rhythm

A practical weekly cadence for the Hack Jam build should use short feedback loops. Monday defines the feature slice and contract. Tuesday creates low-fidelity wireframes and the first technical state model. Wednesday reviews edge cases, permissions and error states. Thursday produces a prototype/POF pair. Friday converts the accepted version into GitHub issues and implementation tasks. This can be compressed

or expanded depending on the sprint week.

Recommended weekly checkpoints

Day	Developer focus	UX/UI focus	Shared output
Monday	Data/state/API questions	User flow and journey	Feature scope
Tuesday	State machine and feasibility	Low-fi wireframe	First contract
Wednesday	Permissions and edge cases	Interaction states	Review fixes
Thursday	Technical POF	Clickable prototype	Validation notes
Friday	GitHub issue + tests	Ready-for-dev frames	Decision log update

7. Miro board structure

Miro should be used as the living systems board rather than as a file dump. Miro supports embedding third-party content such as Figma, Google Docs/Sheets/Slides, Microsoft documents and Power BI content, which makes it useful for connecting research context, flows and design references on one board. [R6]

Recommended Miro zones

- Research evidence: interviews, assumptions, validated/rejected hypotheses.
- Journey maps: buyer, seller, courier, hub operator, transfer operator, moderator, incubation staff.
- Service blueprint: frontstage user action, backstage system event, support process and data object.
- Architecture maps: marketplace, operations, real-time, payment, dispatch and custody flows.
- Open decisions: payment rules, campus hub rules, category-specific fulfilment, staff access and legal/compliance questions.
- Feature backlog map: MVP, P1, research later, institutional decision required.

8. Figma file structure

Figma should be the UX/UI source of truth. Figma Dev Mode gives developers inspection, ready-for-development status, annotations, version comparison and developer integrations; Code Connect can map Figma components to production code components where the team later has the appropriate plan and setup. [R1-R4]

Recommended Figma pages

- 00 - Research references: linked Miro board, assumptions and key user journeys.
- 01 - Wireframes: low-fidelity screens grouped by user role.
- 02 - Marketplace prototype: buyer, seller and courier flows.
- 03 - Operations prototype: hub, transfer, moderation, incubation, finance and admin flows.
- 04 - Components: buttons, cards, inputs, status pills, timers, dispatch cards, QR scanner, map states.
- 05 - Dev annotations: states, API names, error states and acceptance criteria.
- 06 - Archive: rejected concepts and earlier versions.

Figma should not just be screens. It should also include component states. For EduHackers, important states include available, busy, at capacity, paused, pending payment, hub rejected, awaiting repackaging, courier assigned, in transit, transfer exception, investigation open and payout pending.

9. GitHub build workflow

GitHub flow for buildable design work

Issue template minimum:

Problem / hypothesis	Figma link	Backend states
API/event contract	Permissions	Edge cases
Analytics event	Acceptance criteria	Accessability checks

Figure 4. Recommended GitHub workflow. Research and Figma should feed GitHub issues; GitHub should feed implementation, tests and decision records.

GitHub Issues support references between issues and pull requests, and GitHub Projects can plan work using issue metadata and flexible views. [R5] For EduHackers, every buildable feature should become a GitHub issue with links to the Figma frame, Miro evidence and API/state contract.

GitHub issue template minimum

- Feature summary and user role.
- Problem/hypothesis from research.
- Figma link and Miro evidence link.
- Backend state machine or existing states reused.
- API endpoints and WebSocket events.
- Database objects touched.
- Permissions and privacy rules.
- Error/timeout/offline states.
- Acceptance criteria and tests.
- UX review owner and developer review owner.

10. Design system and components

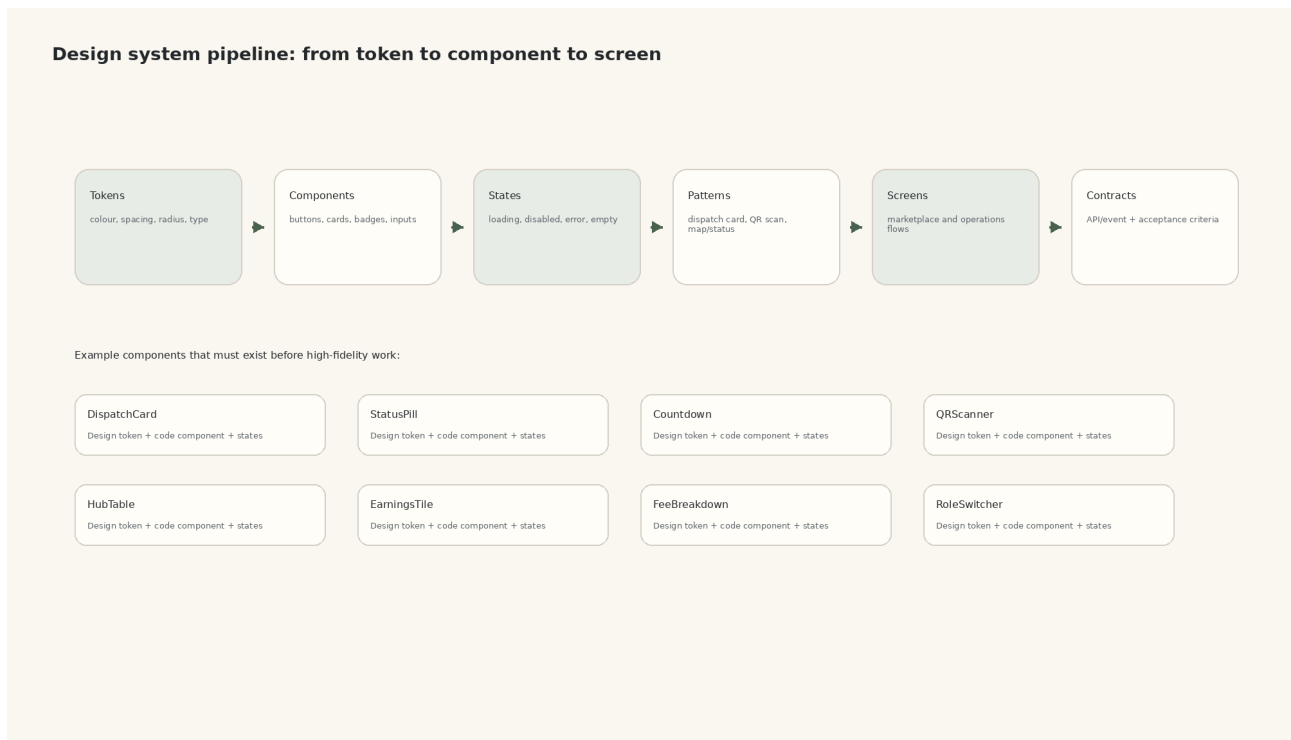


Figure 5. Design system pipeline. EduHackers should define tokens, components, states, patterns and API contracts before polishing final UI.

The team should create a small design system early rather than drawing every screen manually. This reduces inconsistency across Marketplace and Operations apps. The design system should include colour tokens, typography, spacing, radius, elevation, status colours, motion rules, responsive breakpoints and reusable components.

High-risk reusable components

- Dispatch request card with countdown, accept/decline and expired state.
- Status pill system for order, package, transfer, payment and reliability states.
- QR scanning panel with success, mismatch and offline fallback states.
- PIN entry with invalid PIN and retry limits.
- Map / live tracking panel with GPS lost and ETA updated states.
- Fee breakdown component with seller, courier, platform and processor fees.
- Wallet/withdrawal component with spendable, withdrawable and pending amounts.
- Operations table components with audit-friendly history and filter states.

11. Accessibility and inclusive interaction

WCAG 2.2 adds criteria relevant to this product, including focus visibility, target size, dragging movements, redundant entry and accessible authentication. [R7-R8] EduHackers should not treat accessibility as decoration. Timers, QR scanning, maps, live tracking, real-time alerts and complex dashboards create accessibility risk if not planned from wireframe stage.

UX/developer accessibility rules

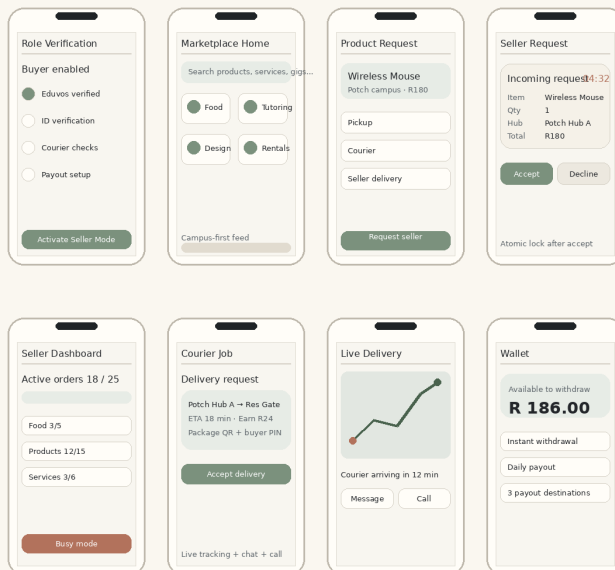
- Countdown timers must have clear text alternatives and should not be communicated only by animation.

- Critical actions such as accepting a request, confirming pickup, entering a PIN and releasing a payout require large touch targets.
- Screens must handle keyboard/focus state for web and desktop clients.
- QR scanning must have fallback codes or manual entry where appropriate.
- Status cannot rely only on colour; use labels and icons/text.
- Maps must have text ETA/status alternatives.
- Forms must avoid requiring repeated entry of the same information when the system already knows it.
- Authentication and step-up verification must not rely on cognitive puzzles alone.

12. Marketplace wireframe examples

EduHackers Marketplace App - low fidelity wireframe examples

Mobile-first flows for buyer, seller and courier modes. These are research wireframes, not final UI.



Developer ↔ UI/UX questions

- What backend state drives each screen?
- What is the loading, timeout, and offline state?
- What action creates a REST call vs a realtime event?
- Which roles can see this screen?
- What happens when a seller/courier is at capacity?
- How does the UI explain protected payment and held funds?
- What accessibility rules apply to timers, alerts, maps and QR scanning?
- Which analytics event proves the flow worked?

Output required before build:

Figma frame + API/state contract + GitHub issue + acceptance criteria

Figure 6. Marketplace app wireframes. These show buyer, seller and courier screens that the UI/UX Designer can convert into polished Figma frames after the Developer confirms state and API contracts.

The marketplace app should support mode switching without creating separate identities. A student can be Buyer-only after Eduvos verification, then activate Seller Mode or Courier Mode through progressive verification. Each role unlocks additional screens and permissions.

Key marketplace screens to design

- Buyer home and campus-first discovery feed.
- Product/service listing and availability state.
- Fulfilment choice: hub pickup, courier delivery, seller delivery or category-specific direct food flow.
- Protected payment and fee breakdown.
- Seller incoming request with staggered batch countdown.
- Seller dashboard with category capacity and overall 25 active-order cap.
- Courier job offer with earnings, pickup/destination zone and ETA.
- Live delivery with tracking, messaging/calling and PIN handoff.
- Wallet and withdrawals with R50 withdrawal minimum and payout destinations.

13. Operations Platform wireframe examples

EduHackers Operations Platform - role-based wireframe examples

One operations app with modules appearing by staff role: Hub, Transfer, Incubation, Moderation, Finance and Admin.

The wireframe shows five distinct modules for different roles:

- Hub Operator: Intake**: Features a QR scan area with a 'SCAN QR' button and a 'Condition: SEALED/INTACT' label. To the right, there are input fields for 'Package ID EH-2041', 'Seller: hidden', 'Category: electronics', and buttons for 'Accept custody' and 'Reject package'.
- Transfer Manifest**: Displays a list of packages from 'Potch Hub → Pretoria Hub' with a '7 / 10 scanned' indicator. The list includes items like 'EH-420 • NORMAL • Pretoria Hub' through 'EH-425 • NORMAL • Pretoria Hub'.
- Moderation: Dispute**: Shows a 'Case: package condition discrepancy' with a list of items: 'Origin: intact', 'Courier scan: intact', 'Destination: damaged', 'Evidence: 3 photos', and 'Action: hold disputed allocation'.
- Incubation Intelligence**: A dashboard with three cards: 'Unmet demand signals' (28), 'Promising sellers' (14), and '30-day plans' (5). Below these is a section for 'AI suggested intervention: marketing support' with a note 'Staff approval required before seller plan.'.
- Admin / Config / Finance**: A configuration page with buttons for 'Campuses & hubs', 'Fee rules', 'Capacity policies', 'Transfer windows', 'Role permissions', and 'Payment exceptions'.

At the bottom, a note states: 'Operations UX principle: expose exactly the operational information needed for the role - no raw buyer/seller private data unless required.' Below this is a flow diagram: 'Hub scan → custody event → condition check → realtime status → audit log → role-specific notification'.

Figure 7. Operations Platform wireframes. One Operations Platform contains role-based modules for Hub Operations, Transfer, Moderation, Incubation, Finance and Admin.

The Operations Platform should be more information-dense than the Marketplace, especially on desktop. Mobile support is still necessary for QR scanning, hub intake and field actions. The same backend identity system should drive staff role permissions.

Key operations screens to design

- Hub intake: scan QR, inspect condition, accept or reject package.
- Rejected package: reason, evidence, 24-hour grace period and seller/buyer notifications.
- Transfer manifest: package count, scan reconciliation, origin/destination custody.
- Dispute workspace: condition history, custody timeline, evidence, statements and payout allocation.
- Incubation dashboard: demand signals, promising sellers, interventions and 30-day plans.
- Finance operations: payment exceptions, refunds, payouts and ledger adjustments.
- Admin configuration: campuses, hubs, role permissions, categories, fees, transfer windows and capacity policies.

14. Detailed flow: seller dispatch

Seller dispatch should be designed as an Uber-style request mechanism adapted to campus commerce. The matching engine ranks eligible sellers, sends staggered batches, locks the order atomically to the first valid accepting seller and cancels other pending offers. UX must show urgency without causing confusion or stress.

Developer states

- REQUEST_CREATED
- SELLER_BATCH_QUEUED
- SELLER_OFFERED
- SELLER_DECLINED
- SELLER_OFFER_EXPIRED
- SELLER_MATCHED_LOCKED
- PAYMENT_PENDING or PAYMENT_AUTHORISED depending on product/service
- FULFILMENT_STARTED

UX questions

- What seller information is essential before Accept?
- How prominent should the countdown be?
- What happens visually if another seller accepts first?
- How is capacity shown when the seller is busy or at limit?
- How does the buyer see alternatives and Notify Me queue if a seller is at capacity?

15. Detailed flow: hub intake and package rejection

Hub intake is a trust-critical workflow. The Hub Operator must scan the package QR, record condition and either accept custody or reject the package before custody is accepted. If rejected, the seller gets a 24-hour correction window, the buyer is notified, and the order remains protected but paused.

Feature contract example

State	UI	Action	Next
HUB_INTAKE_STARTED	QR scan panel	Scan package	CONDITION_CHECK
CONDITION_CHECK	Condition OK?	Accept custody	HUB_RECEIVED
CONDITION_EXCEPTION	Reason + photo required	Reject	AWAITING_SELLER_REPACKAGING
GRACE_EXPIRED	Order correction failed	Auto-cancel	BUYER_REFUND_RELEASED

16. Detailed flow: courier delivery

Courier flows require realtime UI and shared courier capability across categories: live tracking, ETA, order-scoped messaging/calling, earnings, status changes and proof of handoff. Product deliveries generally flow through hubs, while food/perishables can use direct seller-to-buyer pickup when the category policy requires it.

Proof model

- Package QR identifies the package or batch.
- Buyer one-time PIN confirms the correct recipient at final handoff.
- Hub scans establish physical custody.
- Condition checks record package state at custody transfer.

- GPS/live status supports ETA but does not replace scan-based custody evidence.

17. Detailed flow: cross-campus transfer

Cross-campus transfer uses scheduled hub-to-hub batches. Eduvos-approved transport is the primary scheduled backbone; student transfer couriers can provide overflow or additional routes after opting into transfer mode. The UI should hide unnecessary complexity from buyers but show enough status to build trust.

Transfer UX principles

- Buyer sees ETA and current stage: seller hub, in transfer, buyer hub, pickup/local delivery.
- Transfer courier sees package count and minimal package data only.
- Hub Operators scan every package at origin and destination.
- Manifest cannot depart until scan count matches the locked manifest.
- Destination reconciliation controls transfer completion and earnings release.

18. Detailed flow: Incubation intelligence

The Incubation module should not expose mysterious AI scores to sellers. Operations staff can see business health indicators, demand signals, intervention suggestions and approved 30-day plans. Sellers receive practical guidance, not internal scoring. The UX must separate analytics, advice and enforcement.

Developer and UX questions

- What data is safe and useful for Incubation staff?
- Which signals are aggregated rather than individual-level?
- How does staff approve an AI-suggested intervention?
- How does a seller see guidance without seeing a hidden score?
- How do we avoid turning analytics into surveillance?

19. API and event contract approach

OpenAPI defines a standard interface description for HTTP APIs that helps humans and computers understand what a service can do without reading source code or inspecting network traffic. [R9] EduHackers should use OpenAPI for REST contracts and a parallel event schema for WebSocket/realtime events.

Required contract outputs

- REST endpoint list with request/response schemas.
- WebSocket event list with payload schemas.
- Error codes and user-readable messages.
- Permission matrix for each endpoint/event.
- Analytics events mapped to user goals.
- Fixture/test data for UI development before backend is complete.

20. Workflow for wireframes

Wireframes should be produced in levels. Level 0 is the journey map. Level 1 is screen flow and state list. Level 2 is low-fidelity layout. Level 3 is clickable Figma prototype. Level 4 is componentized high-fidelity design. EduHackers should not jump to Level 4 until the state/API contract is stable.

Wireframe checklist

- Role and user goal visible on every frame.
- Entry and exit state named.
- Primary action and fallback action shown.
- Empty/loading/error/offline state planned.
- Notifications and realtime events noted.
- Accessibility notes attached for timers, maps, forms, QR and PIN.
- Linked GitHub issue created when ready for implementation.

21. What the UX/UI Designer should ask the Developer

- What backend states exist for this flow?
- Which states can arrive through WebSocket vs page reload?
- What errors/timeouts are possible?
- What can the user do when the API call fails?
- Which data is private and must not be shown to this role?
- What is the source of truth: database, payment provider, hub scan or location event?
- What is the acceptance test for this flow?
- Which analytics event tells us the design works?

22. What the Developer should ask the UX/UI Designer

- What is the user trying to decide on this screen?
- What information is essential before the user acts?
- How should urgency be communicated without creating stress?
- How does the user recover from a failed state?
- Which states need notifications?
- What should be hidden from this role?
- Which components can be reused?
- How will accessibility be tested?

23. Wireframe-to-build acceptance criteria

A feature should not be marked ready for build until it satisfies these criteria:

- Figma frame exists and is linked to a GitHub issue.
- Miro evidence or research rationale is attached.
- Backend state machine is documented.
- REST and realtime contracts are named.
- Permissions are stated.
- All loading, empty, error, timeout and offline states are accounted for.
- Analytics event exists for success/failure measurement.
- Accessibility checklist has been completed.
- Acceptance criteria can be tested manually and/or in automated tests.

24. Recommended workflow system by team role

Role	Primary tools	Main outputs
Paballo - Developer / Research Lead	GitHub, Miro, API docs, DB model	State machines, API contracts, POF tests, tech decisions
Oratile - UX/UI Designer	Figma, Miro, design system	Wireframes, prototypes, components, interaction states
Product Lead	Miro, Figma, GitHub Projects	Prioritised flows, MVP/P1 scope, acceptance questions
Business Analyst / Marketing	Miro, research docs	User evidence, demand signals, market narrative
Project / Pitch Lead	Slides, Miro, GitHub Projects	Team readouts, pitch structure, progress tracking

25. Risks and mitigations

Risk	Why it matters	Mitigation
Design-build drift	Developer builds different behaviour from Figma	Feature Contract + ready-for-dev review
Over-polished wireframes too early	Team spends time on visual polish before state is known	Low-fi first; contract before high-fi
Hidden backend complexity	UI ignores payment, hub, timeout and error states	State matrix required for every flow
Tool chaos	Miro, Figma and GitHub diverge	Artifact ownership rules and links
Accessibility debt	Timers/maps/QR flows become unusable for some users	WCAG 2.2 checks from wireframe stage
Unclear role access	Operations users see too much or too little	RBAC matrix linked to UI modules

26. Immediate action plan

- Create the Figma file structure and import these low-fidelity wireframe directions as starting frames.
- Create a Miro board section called Developer ↔ UI/UX Contract System.
- Create GitHub issue templates for feature contracts, API contracts and design-ready tasks.
- Select three first flows for POF: seller dispatch, hub intake/rejection, courier live delivery.
- For each selected flow, create one state matrix, one wireframe, one API/event contract and one GitHub issue.
- Run a team review: Product validates problem, UX validates usability, Developer validates feasibility, Project Lead validates pitch clarity.

27. Appendix: example feature contract template

Feature: Seller incoming request. **User role:** Seller. **Goal:** accept or decline a live buyer request quickly. **Entry state:** SELLER_OFFERED. **Primary action:** Accept. **Backend transition:** atomic offer acceptance; order locked; other offers cancelled. **API:** POST /seller-offers/{id}/accept. **Realtime:** seller.matched, buyer.seller_found, seller_offer.closed. **Permissions:** seller must be verified, available, under capacity and associated with listing. **Error states:** expired, already accepted by another seller, capacity exceeded, stock unavailable, network offline. **Acceptance criteria:** two sellers tapping Accept simultaneously results in only one successful lock.

28. References

- R1. Figma Dev Mode overview and developer handoff capabilities. <https://www.figma.com/dev-mode/>
- R2. Figma Guide to Dev Mode: ready-for-development assets, navigation and inspection. <https://help.figma.com/hc/en-us/articles/15023124644247-Guide-to-Dev-Mode>
- R3. Figma Dev Mode code snippets and inspection. <https://help.figma.com/hc/en-us/articles/15023202277399-Use-code-snippets-in-Dev-Mode>
- R4. Figma Code Connect documentation: bridge between Figma components and code components. <https://developers.figma.com/docs/code-connect/>
- R5. GitHub Issues and Projects documentation. <https://docs.github.com/en/issues/tracking-your-work-with-issues/learning-about-issues/about-issues>
- R6. Miro embed integrations documentation. <https://help.miro.com/hc/en-us/articles/360017730993-Inserting-live-content-into-boards>
- R7. W3C WCAG 2.2 Recommendation. <https://www.w3.org/TR/wcag/>
- R8. W3C WCAG overview. <https://www.w3.org/WAI/standards-guidelines/wcag/>
- R9. OpenAPI Specification v3.0.4. <https://spec.openapis.org/oas/v3.0.4.html>